

Real Estate AI Agent Module Documentation

Version: 1.0

Architecture: LangGraph State Machine

Primary Objective: Autonomous, conversational orchestration of real estate operations (Search, Analyze, Book).

1. System Architecture Overview

The `1.Agent` module acts as the "Brain" of the real estate application. It is a cyclical, stateful directed graph built using `langgraph`. Its primary function is to:

1. Parse unstructured human conversations into structured constraints (JSON).
2. Interface with internal predictive models (Recommender & Analyzer) and SQL databases.
3. Synthesize the raw backend outputs into highly polished, elite real-estate consulting narratives.
4. Provide internal quality-assurance guards against AI hallucination and logical drift.

1.1 The Graph Compilation (`4_graph_builder.py`)

The logic flow is instantiated via `build_agent_graph()`.

- **State Dictionary:** Instantiated using `AgentState`.
- **Nodes:**
 - `brain(intent_node)`: Responsible for Natural Language Understanding (NLU).
 - `tools(tool_node)`: Responsible for Backend Execution.
 - `tongue(translator_node)`: Responsible for Natural Language Generation (NLG).
 - `auditor(auditor_node)`: Responsible for Quality Assurance and Safety.
- **Conditional Edges:**
 - **Brain Routing (`route_from_brain_to_next_step`):**
 - `If is_out_of_domain == True OR missing_info is not empty` -> Routes to `tongue` to ask for clarification.
 - `If active_intent is search, analyze, or book` -> Routes to `tools` for execution.
 - **Auditor Routing (`route_from_auditor_to_next_step`):**
 - `If confidence_score >= 0.5` -> Routes to `END` (Completion).
 - `If confidence_score < 0.5` -> Loops back to `brain` for a retry.
- **Memory Checkpointing:** Utilizes `LangGraph's MemorySaver` to persist the state dictionary across conversational turns, allowing multi-turn constraint gathering.

2. Complete State Definition (`3\_state\_definition.py`)

The `AgentState` object is a `TypedDict` that governs the graph. It mutates as it passes through each node.

2.1 Core Conversational Fields

- ``messages`` (`List[Any]`): Annotated with `add_messages`. Holds the sequential dialogue between the user and the AI.
- ``active_intent`` (`str`): The resolved operational mode. Valid values: "search", "analyze", "book", "idle".

2.2 Semantic Extraction Fields

- ``current_filters`` (`dict`): The live dictionary of parsed user constraints.
- *Keys include:* `category` (buy/rent), `town`, `district`, `bedrooms`, `property_type`, `price_max`.
- ``booking_details`` (`dict`): The specific constraints required to book an appointment.
- *Keys include:* `property_id`, `user_name`, `phone`, `date`.
- ``missing_info`` (`List[str]`): Dynamically populated by the Brain. If non-empty, the Tools node is bypassed, and the Tongue asks the user for these specific keys.

2.3 Backend Execution Fields

- ``tool_outputs`` (`dict`): Stores the raw data frames and dictionaries returned by backend services.
- *Keys include:* `recommendations` (from `DataBridge`), `valuation_briefs` (from `FairPriceEstimator`), `market_context` (from `Analyzer`), `booking_response` (from `SQL Server`).

2.4 Control & Safety Fields

- ``confidence_score`` (`float`): 1.0 means verified, 0.0 triggers a graph correction loop.
- ``is_out_of_domain`` (`bool`): Used to intercept and deflect queries unrelated to real estate.
- ``audit_retries`` (`int`): Prevents infinite graph loops. Capped by `AGENT_CONFIG["max_audit_retries"]`.
- ``active_model`` (`str`): Enables "sticky fallbacks". If Groq fails and Gemini succeeds, Gemini becomes the active model for the remainder of the session to prevent repeated latency hits.

3. Deep Dive: Agent Nodes

3.1 The Brain / Intent Node (`nodes/1\_intent\_node.py`)

Goal: Convert string history into a structured JSON payload representing exact user constraints.

Processing Steps:

5. **Context Preparation:** Flattens the LangGraph message list into a formatted `User/AI` string.
6. **LLM Execution Loop:**
 - Invokes the active model (default: `groq/llama-3.3-70b-versatile`) with `INTENT_EXTRACTION_PROMPT`.
 - Forces JSON response format.
 - If the model hallucinates malformed JSON, a 2-try internal loop appends an error message to the prompt and retries.
 - If rate limits hit (429), it injects `__quota_exceeded__` into `missing_info` to trigger a graceful failure.
7. **Entity Regex Safety Net (`extract_entities_via_regex`):**
 - Acts as a deterministic backup.
 - Detects exact terms like "maadi", "zayed", "tagamoa".
 - Translates "7 million" or "7m" to numerical `7000000.0`.
 - Categorizes keywords like "flat" or "apart" -> `Apartment`.
8. **Transaction Inference (`infer_transaction_category`):**
 - Scans the text for `rent`, `lease`, `buy`, `sell`. Uses index distance to resolve conflicts if both terms appear.
9. **State Merging:**
 - Overrides existing `current_filters` with newly extracted LLM filters.
 - Applies the Regex Safety Net **only** to fill in gaps the LLM missed.
10. **Constraint Enforcement:**
 - If `active_intent` is `search`: Asserts that `town`, `property_type`, and `category` must exist. If not, they are added to `missing_info`.
 - If `active_intent` is `book`: Asserts that `property_id`, `user_name`, `phone`, and `date` must exist.

3.2 The Hands / Tool Node (`nodes/2_tool_node.py`)

Goal: Route the validated constraints to the heavy computational modules.

Processing Steps:

11. Search Routing:

- Passes `current_filters` to `DataBridge.execute_recommendation()`.
- **Expert Augmentation:* Loops through the top 3 recommended candidates and proactively calls `DataBridge.execute_fair_price_check()`. This attaches

statistical valuation data (e.g., "Overpriced by 10%") directly to the state for the LLM to use later.

12. Analyze Routing:

- Passes `current_filters` to `DataBridge.execute_analysis("market_pulse")`.

13. Book Routing:

- Passes `booking_details` to `BookingEngine.register_booking()`.

3.3 The Tongue / Translator Node (`nodes/3\_translator.py`)

Goal: Translate raw programmatic states into high-end "Concierge" dialogue.

Processing Steps:

14. Failure Handlers:

- If `__quota_exceeded__` is present, it returns a hardcoded apology.
- If `organic_missing_info` exists, it maps the system keys to human terms (e.g., `category` -> "whether you are looking to buy or rent") and politely asks the user for them.

15. Data Pre-processing:

- Extracts the Top 3 properties from `tool_outputs`.
- Calculates a `budget_fit` percentage (e.g., Target Budget / Property Price).
- Transforms boolean completion statuses into descriptive phrases ("Ready to move in").

16. Narrative Generation:

- Compiles `market_context`, `valuation_briefs`, and the pre-processed properties into a massive contextual prompt.
- Instructs the LLM to categorize the options into tiers ("Primary Match", "Premium Selection", "Value Choice").
- *Correction Loop Payload:* If the Auditor previously failed the response, the Tongue injects the specific error (e.g., "CORRECTION: Do not mention properties outside the given list") to force behavioral correction on this turn.

3.4 The Auditor Node (`nodes/4\_auditor.py`)

Goal: Act as an adversarial network to verify the integrity of the Tongue's output against the raw database facts.

Processing Steps:

17. Execution Gate:

- Automatically skips the audit (sets confidence to 1.0) if the intent is `idle`, the query is out of domain, or the database returned zero results.

18. Context Assembly:

- Creates a strict summary of the database results: `ID, Price, Bedrooms, Area`.

19. Verification Call:

- Feeds the Assistant's proposed response and the strict summary to the LLM with `AUDITOR_SYSTEM_PROMPT`.
- ***Rule - Scale Gate:** Fails the AI immediately if it confuses thousands with millions (e.g., user asks for 10M, AI pitches 900k as the primary match).
- ***Rule - Anti-Generic:** Fails the AI if the database returned 5 properties but the AI responded with a generic "I'll look into it."
- ***Rule - Hallucination:** Fails the AI if it pitches a property ID or price not found in the summary list.

20. Verdict Application:

- Returns JSON: `{ "verdict": "PASS"|"FAIL", "correction_instruction": "..."}.`
- If `FAIL`, it sets `confidence_score` to 0.0, increments `audit_retries`, and injects the `correction_instruction` into `missing_info` so the Tongue can fix it.

4. Sub-System Details

4.1 The Data Bridge (``2_data_bridge.py``)

Acts as the intermediary API surface for the Recommender and Analyzer.

- ``get_grounding_data()``: Queries the `DBAdapter` to pull all distinct `town`, `district`, and `subdistrict` values. Caches this heavily to prevent database lag.
- ``resolve_location(location_name)``: Utilizes `rapidfuzz.process.extractOne` with a `score_cutoff` of 60. This allows users to type "Shorok" and have the system deterministically resolve it to the database standard "Shorouk". It also identifies the hierarchical level (e.g., determining that the string is a `town` versus a `district`).
- ``execute_fair_price_check()``: Wraps the mathematical `FairPriceEstimator.estimate()` and formats it using the `LLMContextBuilder`.
- ``execute_recommendation()``: Normalizes the user's location dictionary (mapping fuzzy inputs to canonical database names) and submits the cleaned dictionary to the `FAISS/XGBoost QueryAdapter`.

4.2 The Booking Engine (``scheduler/booking_logic.py``)

Provides direct, transactional persistence to the SQL Server for appointments.

- **Validation:** Re-checks that `property_id`, `user_name`, `phone`, and `date` exist. Resolves key discrepancies (e.g., `preferred_date` vs `date`).
- **Database Integration:**
 - Uses `pyodbc` with environment variables (`DB_SERVER`, `DB_NAME`, `DB_USER`).
 - Target table: `TblAppointments`.
 - Default Status: Sets `StatusID = 1` (Pending).
 - Data Mapping: Packs `user_name` and `phone` into the CRM Notes column for agent review.
- **Identity Resolution:** Immediately executes `SELECT @@IDENTITY` to return the real database Primary Key to the AI.

4.3 Agent Configuration (``1_agent_config.py``)

A single source of truth for operational tuning.

- `model`: Primary provider (e.g., "groq/llama-3.3-70b-versatile").
- `fallback_models`: List of backups in case of 429 limits or downtime.
- `max_audit_retries`: Hardcoded integer (default: 2). Prevents the LangGraph from hitting infinite recursion limits if the Tongue repeatedly hallucinates.